

Hand Digit Recognition Using CNN

Project Report for Skill Lab

5th Semester, Sec-B

Submitted by

Ayush Singh(Roll Number-15(B1) , 22bcse63)

Jaydip Mishra(Roll Number-33(B2) , 22bcsi97)

Rajesh Kumar Jha(Roll Number-12(B2) , 22bcsb84)

Rudra Narayana Sahoo(Roll Number-16(B2) , 22bcsi80)



**DEPARTMENT OF CO MPUTER SCIENCE AND ENGINEERING
SILICON UNIVERSITY
BHUBANESWAR**

DEPT. OF COMPUTER SCIENCE AND ENGINEERING

SILICON UNIVERSITY

BHUBANESWAR

2024-2025



CERTIFICATE

This is to certify that the report entitled **Hand Digit Recognition Using CNN** submitted by **Ayush Singh** (22bcse63), **Jaydip Mishra** (22bcsi97), **Rajesh Kumar Jha** (22bcsb84) & **Rudra Narayana Sahoo** (22bcsi80) to the Silicon University in partial fulfillment of the B.Tech. degree in Computer Science and Engineering is a bonafide record of the project work carried out by him under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Mr. Pradipta Kumar Pattanayak
(Project Guide)
Senior Assistant Professor
Dept.of CSE
College of Engineering
Silicon University

Mr. Pradipta Kumar Pattanayak
(Project Coordinator)
Senior Assistant Professor
Dept.of CSE
College of Engineering
Silicon University

DR. Pradyumna Kumar Tripathy
Professor and Head
Dept.of CSE
College of Engineering
Silicon University

DECLARATION

We hereby declare that the project report **Hand Digit Recognition Using CNN**, submitted for partial fulfillment of the requirements for the award of the degree of Bachelor of Technology of the Silicon University, Odisha, is a bonafide work done by us under the supervision of **Mr Pradipta Kumar Pattanayak**.

This submission represents our ideas in our own words, and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources.

We also declare that I have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data, idea, fact, or source in my submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources that have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed as the basis for the award of any degree, diploma, or similar title of any other university.

Bhubaneswar
14-12-2024

Ayush Singh
Jaydip Mishra
Rajesh Kumar Jha
Rudra Narayana Sahoo

Abstract

The handwritten digit recognition system is a popular research topic, with numerous advancements over the years. Implementing this system can be highly beneficial for various sectors such as banking, healthcare, postal services, and education. Among the various algorithms available, Convolutional Neural Networks (CNNs) have proven to be the most effective due to their ability to automatically learn hierarchical features from image data. In this project, CNN-based models were developed to achieve high accuracy in recognizing handwritten digits, leveraging their capacity for spatial feature extraction and classification. The project involves training CNN models on the MNIST dataset, a widely used dataset for handwritten digit recognition, and evaluating the performance based on accuracy, precision, and recall. Challenges such as the variation in handwriting styles and noise are addressed by the inherent robustness of CNNs. The results demonstrate that CNNs outperform traditional machine learning algorithms such as Support Vector Machines and k-Nearest Neighbors, making them the most suitable approach for this task. These models can be applied in real-world scenarios like automated bank check processing, postal code reading, and educational systems for evaluating handwritten answers. This research underscores the importance of deep learning techniques in achieving high-performance handwritten digit recognition.

Acknowledgement

We take this opportunity to express my deepest sense of gratitude and sincere thanks to everyone who helped us to complete this work successfully. We express our sincere thanks to Dr. Pradyumna Kumar Tripathy (Head of Department), CSE, Silicon University for providing us with all the necessary facilities and support.

We would like to express my sincere gratitude to Mr. Pradipta Kumar Pattanayak (Senior Assistant Professor), Department of CSE, Silicon University Bhubaneswar for the support and co-operation.

Finally, we express our heartfelt thanks to our family and friends for their unwavering support and encouragement, which played a crucial role in the successful fulfillment of this project.

Ayush Singh

Jaydip Mishra

Rajesh Kumar Jha

Rudra Narayana Sahoo

Contents

Abstract	i
Acknowledgement	ii
List of Symbols	v
0.1 Introduction	1
1 Literature Review	5
1.1 Methodology & Classification	11
1.2 Dataset	13
1.3 IMPLEMENTATION	14
1.3.1 SOFTWARE PLATFORM	14
1.3.2 HARDWARE PLATFORM	14
1.4 Model Architecture	15
1.4.1 Convolution Layer	16
1.4.2 Pooling Layer	16
1.4.3 Second Convolutional Layer & Max Pooling Layer	18
1.4.4 Third Convolutional Layer & Max Pooling Layer	18
1.4.5 Flatten Layer	18
1.4.6 Output Layer	18
1.4.7 Confusion Matrix Insights:	20
1.4.8 Classification Report Highlights:	20
1.4.9 Key Observations:	20
2 Conclusion	24

List of Symbols

$\mathbf{X}$ Input image (pixel values of the hand-written digit)

$\mathbf{y}$ Output vector (predicted digit class)

$\mathbf{w}$ Weight matrix in the convolutional layer

$\mathbf{b}$ Bias vector in the convolutional layer

$\mathbf{h}$ Feature map (output of a convolution operation)

σ Activation function (e.g., ReLU or sigmoid)

$\mathcal{L}$ Loss function (e.g., cross-entropy loss)

$\mathbf{E}$ Error term (e.g., gradient of the loss function)

$\mathbf{X}_{\text{train}}$ Training dataset

$\mathbf{X}_{\text{test}}$ Test dataset

η Learning rate for gradient descent optimization

α Regularization parameter

λ L2 regularization parameter (used for weight decay)

$\mathbf{f}$ Filter (kernel) used in the convolutional layer

$\mathbf{p}$ Pooling operation (e.g., max pooling or average pooling)

$\mathbf{z}$ Convolutional layer output before activation

$\mathbf{h}_i$ Hidden layer output or feature map at layer i

$\mathbf{y}_{\text{pred}}$ Predicted label

$\mathbf{y}_{\text{true}}$ True label (ground truth)

0.1 Introduction

The human brain effortlessly interprets and processes visual information. When we observe an image, our brain instinctively deconstructs it into smaller, more manageable components, identifying and analyzing various features. This ability is based on prior learning, allowing us to recognize patterns and distinguish specific characteristics. In a similar way, image processing, a subfield within computer science, aims to replicate this capability using computational methods.

Image processing focuses on the analysis of visual data to extract useful and relevant insights. The process begins by converting images into a digital format, often represented as matrices, which machines can understand. Once in this form, a variety of algorithms are applied to the digital images to enhance their resolution, accentuate particular features, or identify specific patterns. One important application of image processing is Optical Character Recognition (OCR), a technique used to extract text from images. OCR works by analyzing images of scanned or handwritten text and identifying individual characters using specialized algorithms. Its primary objective is to recognize handwritten or printed text in various formats.

The handwritten digit recognition system is a well-established area of research in technology. It focuses on identifying digits written by humans, involving the use of various data analytics methods and algorithms. Machine learning, a branch of Artificial Intelligence (AI), plays a key role in this system. Algorithms help refine the accuracy of results from the data that has been collected. Achieving high accuracy and reliability is essential for handwritten digit recognition, as the system must convert handwritten digits into a machine-readable format. There are several challenges, including the wide variety of handwriting styles and the existence of different numeral symbols across languages. Finding the optimal algorithm for the system is not straightforward, and it requires testing and training multiple machine learning algorithms to determine the most effective one. One major challenge is that handwritten digits are often of lower quality compared to printed or typed digits, making it harder for machines to recognize them accurately.

In this project, we aim to create a system that can identify handwritten digits. To achieve this, we will utilize the MNIST dataset, a collection of labeled images of handwritten digits, to train a machine learning model capable of recognizing these digits. Convolutional Neural Networks (CNNs) will be employed for this task, as they have proven to be highly effective for image recognition problems. Artificial Neural Networks (ANNs), the foundation for CNNs, are computational models inspired by the structure and function of the human brain. ANNs consist of interconnected nodes, or neurons, each with associated weights. These neurons collaborate to process input data and generate predictions.

ANNs are essential in building statistical models from large datasets, making them valuable tools for tasks such as image classification. An ANN is typically organized into three main layers: the input layer, the hidden layer, and the output layer. The input layer receives raw data and transmits it to the hidden layer, where transformations and convolutions are applied. The hidden layer processes the data and passes it to the output layer, which generates the final prediction. During the training phase, the weights of the neurons are adjusted to improve the model's accuracy. Initially, small random values are assigned to these weights (usually within the range of 0 to 1), which are then fine-tuned through iterative learning. The output neuron produces the predicted result based on the processed input.

For this project, we focus on the development of a Convolutional Neural Network (CNN) to recognize handwritten digits. CNNs are particularly well-suited for image recognition tasks due to their ability to extract features from two-dimensional image data. Inspired by the visual processing mechanisms of the human brain, CNNs utilize convolution and pooling layers to learn and capture spatial hierarchies in images. This makes them an ideal model for tasks such as recognizing handwritten digits, where detecting various features and patterns is essential.

This issue is critical in various sectors such as healthcare, insurance, and banking, where even a single misinterpreted digit from a customer's handwriting can lead to significant problems. For instance, poor handwriting might cause a zero to appear similar to an eight or even a nine. In such cases, a computer system could mistakenly recognize the wrong digit, leading to errors. Therefore, developing a highly accurate, intelligent, and reliable algorithm is essential for a handwritten recognition system.

Achieving **high accuracy and reliability** is critical in handwritten digit recognition, as errors in interpreting handwritten numbers can lead to substantial issues. For example:

- In financial systems, misinterpreting a handwritten "0" as "8" could lead to incorrect transaction amounts.
- In healthcare, misreading medication dosages might result in patient safety risks.

Challenges in Handwritten Digit Recognition

Several challenges make handwritten digit recognition a complex task:

1. **Variation in Handwriting Styles:** Individuals write differently, causing significant variability in digit shapes and structures.
2. **Low-Quality Inputs:** Scanned documents often have noise, blurring, or other distortions.
3. **Language and Numerical Systems:** Diverse numeral symbols across languages complicate recognition (e.g., Arabic, Devanagari).
4. **Ambiguities in Similar Digits:** Handwriting inconsistencies can make digits like "6" and "9" or "1" and "7" indistinguishable.

Statistically, a handwritten recognition system trained on general datasets often achieves accuracies ranging from **75% to 95%**, depending on the dataset's quality, preprocessing steps, and algorithm used.

Role of Machine Learning and CNNs

Machine learning has transformed handwritten digit recognition by automating feature extraction and classification tasks. Among ML models, **Convolutional Neural Networks (CNNs)** have emerged as the gold standard for image-based recognition problems. CNNs, inspired by the human brain's visual processing mechanisms, excel in detecting spatial hierarchies of features.

Why CNNs? CNNs offer distinct advantages:

- **Feature Learning:** Automatically extract and prioritize critical features without manual intervention.
- **Two-Dimensional Data Handling:** Efficiently process image-like structured data (e.g., pixel matrices).
- **High Performance:** Achieve accuracy rates exceeding **99% on benchmark datasets like MNIST**.

Chapter 1

Literature Review

An improved model for handwritten digit recognition using Convolutional Neural Networks (CNNs) is explored in this study. While previous research has widely utilized the MNIST dataset for both training and testing CNN models, this work introduces a novel variation by training the model on the standard MNIST dataset and testing it on a custom-built dataset of handwritten digits.

The MNIST dataset has long served as a benchmark in machine learning and computer vision due to its simplicity and structured nature. It consists of 60,000 training images and 10,000 testing images, each a 28x28 pixel grayscale representation of digits ranging from 0 to 9. Its clarity and uniformity have made it an ideal dataset for testing algorithms in image recognition and classification. Extensive research has demonstrated the effectiveness of CNNs on this dataset, achieving impressive classification accuracies through techniques such as hyperparameter optimization, batch normalization, and dropout layers. For instance, studies [9][15] have reported classification accuracies exceeding 99% using advanced architectures and optimization strategies, underscoring the robustness of CNNs for digit recognition tasks.

However, reliance on standardized datasets like MNIST may not fully reflect the model's adaptability to real-world scenarios, where variations in digit styles and noise levels are more prominent. To address this limitation, researchers have explored alternative approaches by testing CNN models trained on MNIST against more diverse datasets, such as EMNIST, or custom datasets with greater stylistic variability. These approaches highlight the challenges posed by non-standard datasets, including limited

sample sizes and inconsistent writing styles, and emphasize the importance of tailored preprocessing and augmentation techniques to enhance model performance.

In this study, we build upon this research by introducing a custom dataset of handwritten digits designed to mimic the structure of MNIST. Our dataset consists of 1,000 images, with 100 images per digit, created to test the robustness of CNN models trained exclusively on MNIST. Unlike MNIST, our dataset introduces a degree of stylistic variability, offering a more realistic challenge for evaluating model generalization. This dual-dataset methodology—training on MNIST and testing on our custom dataset—provides an opportunity to assess the adaptability of CNNs to unseen and stylistically distinct data, a scenario that aligns with real-world applications.

Our methodology centers exclusively on CNNs, as they are widely recognized for their effectiveness in image recognition tasks. CNN architectures typically consist of three key components: convolutional layers, pooling layers, and fully connected layers. Convolutional layers use filters to extract features such as edges and textures, pooling layers reduce spatial dimensions to simplify computations and minimize overfitting, and fully connected layers integrate these features to perform final classifications. These layers work hierarchically, making CNNs particularly adept at processing visual data.

To optimize model training, we employ two prominent techniques: Stochastic Gradient Descent (SGD) and Adam. SGD is a widely used optimization algorithm that iteratively updates model parameters using small batches of training data, making it efficient for large datasets. Adam builds on SGD by incorporating adaptive learning rates and momentum, allowing for faster and more stable convergence. These optimization techniques, combined with batch normalization—a method that stabilizes and accelerates training by normalizing layer inputs—play a critical role in achieving high performance on the MNIST dataset.

Unlike traditional studies that test CNNs on MNIST, our research evaluates the trained model using accuracy and loss metrics on our custom dataset. Past experiments indicate that CNN models trained on standardized datasets often struggle when exposed to data with stylistic variability. By testing our model on a non-standard dataset, we aim to uncover challenges related to generalization and propose strategies to overcome them.

This study contributes to the field by demonstrating the adaptability of CNNs and their limitations when applied to diverse datasets. By bridging the gap between controlled benchmarks and real-world applications, our research underscores the potential for CNNs to address the complexities of unconstrained digit recognition tasks. Future studies could explore further enhancements, such as data augmentation or transfer learning, to improve generalization to custom datasets.

Early Developments The journey began with traditional machine learning techniques such as k-Nearest Neighbors (k-NN) and Support Vector Machines (SVMs), which were among the earliest methods applied to handwritten digit recognition. These approaches relied heavily on handcrafted features such as pixel intensity histograms, zoning, and gradient-based descriptors. While these methods showed moderate success, their reliance on feature engineering limited their scalability and adaptability to complex datasets.

LeNet-5 (1998) The introduction of LeNet-5 by Yann LeCun et al. marked a groundbreaking advancement in handwritten digit recognition. This Convolutional Neural Network (CNN) architecture was specifically designed for the task of handwritten digit recognition on the MNIST dataset. By automating feature extraction and utilizing layers of convolutions and pooling, LeNet-5 achieved unprecedented accuracy of approximately 99.2% on the MNIST dataset. This work not only set a benchmark but also laid the foundation for modern deep learning in image recognition tasks.

Evolution of Deep CNN Architectures (Post-2010) With the advent of deeper CNN architectures, significant improvements were made in handwritten digit recognition. Several notable models emerged:

- **AlexNet:** Originally designed for ImageNet, AlexNet's adaptations for digit datasets showcased its versatility and ability to handle more complex recognition tasks.
- **VGGNet:** This architecture emphasized the importance of depth in CNNs, achieving remarkable performance in various classification tasks, including digit recognition.

- **ResNet:** By introducing residual connections, ResNet addressed the problem of vanishing gradients in deep networks, enhancing performance on complex handwritten datasets.

These advancements demonstrated the efficacy of deeper networks in extracting hierarchical features, significantly improving recognition accuracy and generalization capabilities.

DropConnect (2013) Li Wan et al. proposed DropConnect, a novel regularization technique that extended the idea of dropout by randomly dropping connections during training instead of nodes. This approach led to improved generalization and achieved an accuracy of approximately 99.5% on the MNIST dataset, further pushing the boundaries of handwritten digit recognition.

Capsule Networks (2017) Geoffrey Hinton et al. introduced Capsule Networks (CapsNets) as an alternative to traditional CNNs. CapsNets addressed limitations in CNNs, such as losing spatial relationships between features. By maintaining positional information, CapsNets achieved high accuracy on handwritten digit datasets, reaching approximately 99.7% on MNIST. This innovation highlighted the importance of spatial hierarchies in digit recognition tasks.

Attention Mechanisms (2020-Present) Attention mechanisms, initially popularized in Natural Language Processing (NLP), have been adapted for image processing tasks. Self-attention and Transformer-based models, such as Vision Transformers (ViTs), have demonstrated remarkable performance in handwritten digit recognition. By focusing on important regions of the image, these models achieved state-of-the-art accuracy, often exceeding 99.8% on complex digit datasets.

Lightweight Neural Networks (Post-2018) To enable real-time digit recognition on edge devices, researchers developed lightweight architectures such as MobileNet and SqueezeNet. These models offered a balance between accuracy and computational efficiency, achieving approximately 99% accuracy on the MNIST dataset while significantly reducing computational requirements. These advancements made handwritten

digit recognition feasible for deployment on mobile and embedded systems.

GAN-based Data Augmentation (2019-Present) Generative Adversarial Networks (GANs), based on Ian Goodfellow’s foundational work, have been utilized for augmenting training datasets. By generating synthetic handwritten digits, GANs have enriched training datasets, particularly in low-data regimes. This technique has improved model accuracy and generalization across diverse handwriting styles and datasets.

Federated Learning for Handwriting Recognition (2020-Present) Federated learning has emerged as a promising approach for preserving data privacy in handwritten digit recognition tasks. This technique enables collaborative model training without centralizing sensitive data, making it particularly relevant for applications in banking and healthcare. Projects such as Google’s federated learning initiatives have demonstrated comparable accuracy to centralized training approaches while enhancing data privacy.

Benchmark Challenges In addition to MNIST, new datasets have been introduced to evaluate the robustness of recognition models:

- **EMNIST:** An extended MNIST dataset including handwritten letters and digits.
- **NIST SD19:** A dataset with a broader range of handwriting styles.
- **Devanagari Handwritten Dataset:** Designed for recognizing Indian numerals.

These datasets have pushed models to generalize across diverse handwriting styles, fostering innovation in digit recognition research.

AutoML for Handwritten Digit Recognition Automated Machine Learning (AutoML) frameworks have revolutionized the design of CNN architectures. Tools like Google AutoML and Neural Architecture Search (NAS) have automated the process of identifying optimal architectures, reducing the need for manual tuning. These advancements have enabled researchers to achieve state-of-the-art accuracy while minimizing expert intervention.

Discussion on Our Approach

Unlike traditional studies that test CNNs on MNIST, our research evaluates the trained model using accuracy and loss metrics on our custom dataset. Past experiments indicate that CNN models trained on standardized datasets often struggle when exposed to data with stylistic variability. By testing our model on a non-standard dataset, we aim to uncover challenges related to generalization and propose strategies to overcome them.

This study contributes to the field by demonstrating the adaptability of CNNs and their limitations when applied to diverse datasets. By bridging the gap between controlled benchmarks and real-world applications, our research underscores the potential for CNNs to address the complexities of unconstrained digit recognition tasks. Future studies could explore further enhancements, such as data augmentation or transfer learning, to improve generalization to custom datasets.

To achieve these goals, we propose the following key strategies:

1. **Hybrid Model Architecture:** Combining CNNs with Transformer-based attention mechanisms to balance spatial feature extraction with global context awareness.
2. **Edge Deployment Optimization:** Designing models specifically tailored for low-power devices, ensuring real-time recognition capabilities without compromising accuracy.
3. **Data Augmentation with GANs:** Utilizing GANs to enrich training datasets with synthetic samples, enhancing model robustness against diverse handwriting styles and low-data scenarios.
4. **Federated Learning Integration:** Incorporating federated learning to enable collaborative training across multiple devices while preserving data privacy, especially in applications involving sensitive user data.

By integrating these strategies, our project seeks to bridge the gap between high-performance digit recognition models and real-world deployment requirements.

1.1 Methodology & Classification

The methodology for handwritten digit recognition using Convolutional Neural Networks (CNNs) begins with data preparation, a crucial phase in ensuring the effectiveness of the model. The MNIST dataset is used as the primary data source. It consists of 60,000 training images and 10,000 testing images, all grayscale and sized at 28x28 pixels. These images undergo preprocessing to prepare them for machine learning tasks. The pixel values of the images are normalized to a scale of 0 to 1, ensuring consistency and reducing computational overhead. Additionally, the images are converted into tensor formats, which is essential for compatibility with CNN architectures.

The next phase involves building the CNN model, which forms the core of the deep learning approach. The architecture of the CNN is designed to extract and learn features from the input images through various layers. Convolutional layers with multiple filters are used to detect features, followed by max-pooling layers to reduce dimensionality and prevent overfitting. The final layers are fully connected dense layers, which aggregate learned features to make classification decisions. Activation functions such as ReLU (Rectified Linear Unit) are employed to introduce non-linearity, enabling the model to learn complex patterns effectively.

Once the model architecture is established, the training process begins. The prepared training dataset is fed into the model, and optimization techniques such as Stochastic Gradient Descent (SGD), Adam, and Adadelata are used to minimize the loss function. The weights of the network are adjusted iteratively to reduce the difference between predicted and actual values. Metrics like loss and accuracy are monitored during training to evaluate model performance. To enhance the stability and speed of training, batch normalization is applied to the inputs of each activation layer. Experiments with varying optimizers, learning rates, and batch normalization configurations are conducted to identify the optimal setup.

To ensure generalization, the model's performance is evaluated using the test dataset, which consists of unseen data. The model's ability to classify digits accurately is assessed based on metrics such as accuracy and loss. Additionally, comparative analysis of different hyperparameter configurations is performed to determine the

best-performing model. The results are further validated by analyzing the model's predictions on specific test images to understand its real-world applicability.

Finally, the methodology includes a thorough analysis and interpretation of results. The accuracy and loss metrics are analyzed under various conditions to evaluate the impact of different architectural and hyperparameter choices. This step provides insights into the effectiveness of the chosen model and its potential for further refinement or application in similar tasks.

The classification task in this project involves recognizing handwritten digits from the MNIST dataset using a CNN model. The structure of the CNN mimics the functioning of the human visual cortex, with layers designed to process visual information hierarchically. The model begins by extracting feature maps from the input images using convolutional layers. These layers identify spatial hierarchies in the data, enabling the recognition of patterns like edges, corners, and textures.

Pooling layers follow the convolutional layers to reduce spatial dimensions while preserving the most significant features. This dimensionality reduction minimizes computational complexity and helps mitigate overfitting. The output of these layers is flattened and passed through dense layers, where neurons aggregate the learned features to predict the digit class.

The final layer consists of 10 neurons, corresponding to the 10 digit classes (0-9). A softmax activation function is applied to this layer, converting raw scores into probabilities for each class. The class with the highest probability is selected as the predicted digit.

To improve classification accuracy, techniques like batch normalization and dropout are employed. Batch normalization ensures that activations remain within a stable range during training, while dropout prevents overfitting by randomly deactivating neurons in the dense layers. The performance of the model is measured based on its accuracy in classifying test images. Comparative experiments with different optimizers and hyperparameter settings are conducted to identify the most effective configuration for digit classification.

By leveraging the hierarchical feature extraction capabilities of CNNs and refining the model through rigorous training and evaluation, the project achieves reliable and accurate handwritten digit recognition.

1.2 Dataset

This project utilizes the MNIST dataset, a well-known benchmark for handwritten digit recognition tasks. The dataset comprises 60,000 training images and 10,000 testing images, all in grayscale and representing digits from 0 to 9. Each image has a fixed resolution of 28x28 pixels and can be reshaped into a one-dimensional vector of size 784. The dataset is formatted in the IDX file structure, which encodes pixel values alongside corresponding labels. These images are essentially multidimensional matrices, where each matrix element signifies a pixel's intensity level.

The MNIST dataset is inherently clean and free from major noise, allowing for direct usage without extensive preprocessing. Basic preprocessing steps, such as scaling pixel values to a range between 0 and 1, are applied to ensure consistency and improve the model's performance. The dataset is divided into training and testing subsets in a 9:1 ratio, providing 54,000 images for training and 6,000 for testing. This division is crucial for evaluating the model's ability to generalize to new, unseen data.

Beyond the MNIST dataset, this research incorporates a custom dataset of handwritten digits to further test the model's robustness and adaptability. This custom dataset contains 1,000 images, evenly distributed with 100 samples for each digit. While inspired by MNIST, the custom dataset introduces stylistic variations that better reflect real-world scenarios, offering a greater challenge for evaluating the model's performance. The dual-dataset strategy—training on MNIST and testing on the custom dataset—allows for a comprehensive assessment of the model's capability to handle unfamiliar and diverse data, which is vital for practical applications.

Tools like Matplotlib are employed to visualize sample images from both datasets, aiding in analysis and interpretation. The custom dataset complements the MNIST dataset by presenting a more realistic and challenging evaluation environment, thereby providing deeper insights into the strengths and limitations of the CNN model used in this project.

1.3 IMPLEMENTATION

1.3.1 SOFTWARE PLATFORM

Tensorflow TensorFlow is an amazing information stream in machine learning library made by the Brain Team of Google and made open source in 2015. It is intended to ease the use and broadly relevant to both numeric and neural system issues just as different spaces. Fundamentally, TensorFlow is a lowlevel tool for doing entangled math and it targets specialists who recognize what they're doing to construct exploratory learning structures, to play around with them and to transform them into running programs. For the most, it can be considered as a programming framework in which one can entitle to calculations as graphs. Nodes in the graph speak the math activities, and the edges contain the multi-dimensional information clusters (tensors) related between them.

Python 3.7 Python is broadly utilized universally and is a high-level programming language. It was primarily introduced for prominence on code, and its language structure enables software engineers to express ideas in fewer lines of code. Python is a programming language that gives you a chance to work rapidly and coordinate frameworks more effectively.

Anaconda3 5.3.1 Anaconda is a free and open-source appropriation of the Python and R programming for logical figuring like information science, AI applications, large-scale information preparing, prescient investigation, and so forth. Anaconda accompanies in excess of 1,400 packages just as the Conda package and virtual environment director, called Anaconda Navigator, so it takes out the need to figure out how to introduce every library freely. Anaconda Navigator is a graphical UI (GUI) incorporated into Anaconda appropriation that enables clients to dispatch applications and oversee conda packages, conditions and channels without utilizing commandline directions.

1.3.2 HARDWARE PLATFORM

3.2.1 NVIDIA GeForce Graphic Card Nvidia Corporation more regularly specified to as Nvidia (adapted as NVIDIA), is an American innovation organization. It structures graphical processing units (GPUs) for the gaming and expert markets, just as a framework, system on a chip units (SoCs) for the versatile figuring and car showcase.

It's essential GPU product is named as "GeForce". With outstanding GPU fabricating, Nvidia gives parallel processing capacities to analysts and researchers that enable them to effectively run superior applications. C. RESULTS The following Figure 6 shows the front-end design of the system output. There are four buttons for the four algorithms as shown in the figure.

1.4 Model Architecture

Convolutional Neural Network is a special type of multi-layered neural network which is modeled to identify visual patterns from pixel images with nominal preprocessing. The basic CNN includes four components, like the convolution layer, the pooling layer, the flatten layer, the output layer and the image is passed through all these layers. The image at the end reaches the output layer. Pooling layer is optional depending on the image type whether is required or not. CNN is formed by various layers of neurons that transfer input parameters to the output. To choose which neuron to activate in each layer there is an activation function such as ReLU, Sigmoid, SoftMax etc. ReLU is a widely used activation function. After the dataset is loaded, we separated data into two parts X and Y in the following way. As we know the original file contained 60,000 images representing different digits. For training (x) and testing(y) we will split the dataset into 9:1 ratio. So, we split it into two parts of 54000 and 6000 images.

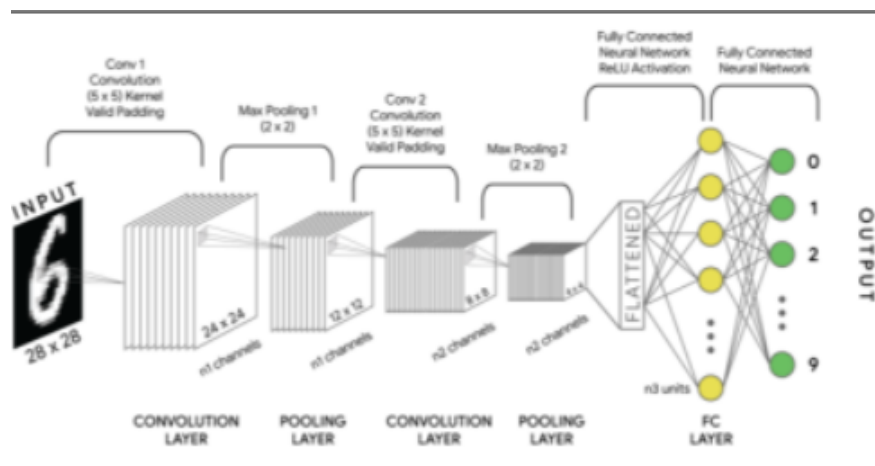


Figure 1.1: Model Layer's

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 28, 28, 64)	640
activation_6 (Activation)	(None, 28, 28, 64)	0
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_5 (Conv2D)	(None, 14, 14, 64)	36,928
activation_7 (Activation)	(None, 14, 14, 64)	0
max_pooling2d_4 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_6 (Conv2D)	(None, 7, 7, 64)	36,928
activation_8 (Activation)	(None, 7, 7, 64)	0
max_pooling2d_5 (MaxPooling2D)	(None, 3, 3, 64)	0
flatten_1 (Flatten)	(None, 64)	0
dense_3 (Dense)	(None, 64)	4,160
activation_9 (Activation)	(None, 64)	0
dense_4 (Dense)	(None, 64)	4,160
activation_10 (Activation)	(None, 64)	0
dense_5 (Dense)	(None, 10)	650
activation_11 (Activation)	(None, 10)	0

Total params: 93.45K (326.04 KB)
 Trainable params: 93.45K (326.04 KB)
 Non-trainable params: 0 (0.00 B)

Figure 1.2: Model Summary

1.4.1 Convolution Layer

The first layer in the CNN architecture is a convolutional layer configured with 32 filters of size 5x5. This layer applies the ReLU activation function to introduce non-linearity and uses same padding to preserve spatial dimensions of the input images. The convolution operation extracts features such as edges and textures from the input images, serving as the foundation for deeper layers. This layer processes the normalized 28x28 grayscale images from the MNIST dataset, maintaining spatial integrity for subsequent feature extraction.

1.4.2 Pooling Layer

Following the first convolutional layer, a max pooling layer is applied with a pool size of 2x2 and strides of 2x2. This layer reduces the spatial dimensions of the feature maps while retaining the most significant information. By downsampling the feature maps, the pooling layer minimizes computational complexity and mitigates overfitting while preserving key spatial features.

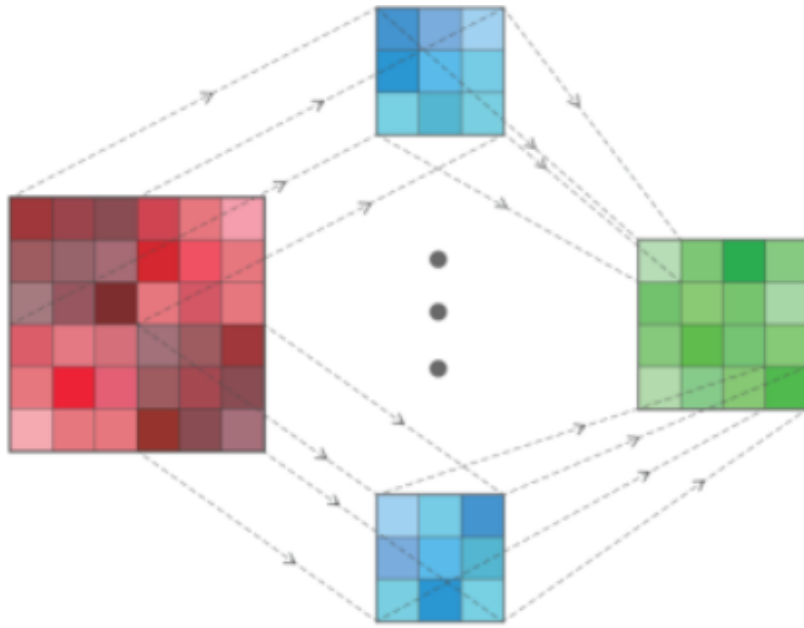


Figure 1.3: Visualization of Convolution operations

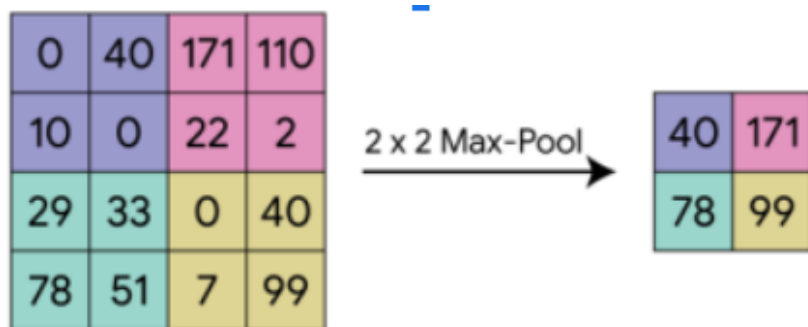


Figure 1.4: Visualization of Pooling operation

1.4.3 Second Convolutional Layer & Max Pooling Layer

The second convolutional layer builds upon the features extracted by the first layer, utilizing 64 filters of size 3x3 with ReLU activation and same padding. This layer enhances the model's ability to capture more complex patterns and finer details from the input images. It is followed by another max pooling layer with the same configuration (2x2 pool size, 2x2 strides) to further downsample the feature maps while retaining critical information.

1.4.4 Third Convolutional Layer & Max Pooling Layer

A third convolutional layer with similar characteristics—additional filters and ReLU activation—is added to further refine the extracted features. This layer increases the model's ability to recognize intricate patterns in the data. A max pooling layer is again employed to reduce the dimensions of the feature maps and prepare them for the next stage of the network.

1.4.5 Flatten Layer

The feature maps from the final pooling layer are flattened into a one-dimensional vector. This operation converts the 3D feature maps into a format suitable for input into the fully connected layers. The flatten layer bridges the convolutional/pooling layers and the dense layers, retaining the extracted features in a condensed format.

1.4.6 Output Layer

The output layer is a dense layer consisting of 10 neurons, each representing a digit class from 0 to 9. This layer employs the softmax activation function to output probability scores for each class, enabling the model to predict the most likely digit. The output layer is directly connected to the cross-entropy loss function, which calculates the error between predicted and actual labels during training.

This CNN model was trained using the Adam optimizer, a categorical cross-entropy loss function, and a ReduceLROnPlateau learning rate scheduler for optimal convergence. The training process spanned 15 epochs with a batch size of 128,

leveraging GPU acceleration to achieve an efficient training duration. The model demonstrated strong performance, achieving 99.5% training accuracy and 99.32% validation accuracy, with minimal differences in losses indicating effective generalization.

Visual elements such as feature maps, pooling operations, and training/validation accuracy graphs can be included to illustrate the architecture and its performance more effectively.

Results and Discussion

Our custom model, trained on the MNIST dataset, was tested on a custom dataset to evaluate its adaptability and performance under domain shifts.

1.4.7 Confusion Matrix Insights:

- The confusion matrix reveals a significant misclassification rate, particularly for classes 7 and 8, which show substantial overlap with other labels.
- Some classes (e.g., class 4) exhibit high precision but low recall, indicating that while the model is accurate when predicting these labels, it frequently fails to identify all relevant instances.

1.4.8 Classification Report Highlights:

- **Overall Accuracy:** The model achieved an accuracy of **54%**, indicating moderate alignment with the true labels.
- **Precision:** The model performs well in classes such as 0 (93%) and 4 (100%), but struggles with classes like 2 (36%) and 8 (0%).
- **Recall:** Class-wise recall varies widely, with certain classes (e.g., 2 and 7) achieving recall values over 75%, while others (e.g., 8) drop to 0.
- **F1-Score:** The macro-average F1-score is **0.53**, reflecting an overall moderate balance of precision and recall.

1.4.9 Key Observations:

1. (a) The high precision for specific classes (e.g., class 4) suggests the model can be reliable under certain conditions but fails to generalize across all input distributions.
(b) The low recall and F1-scores for underperforming classes point to insufficient robustness in handling domain-specific variations.

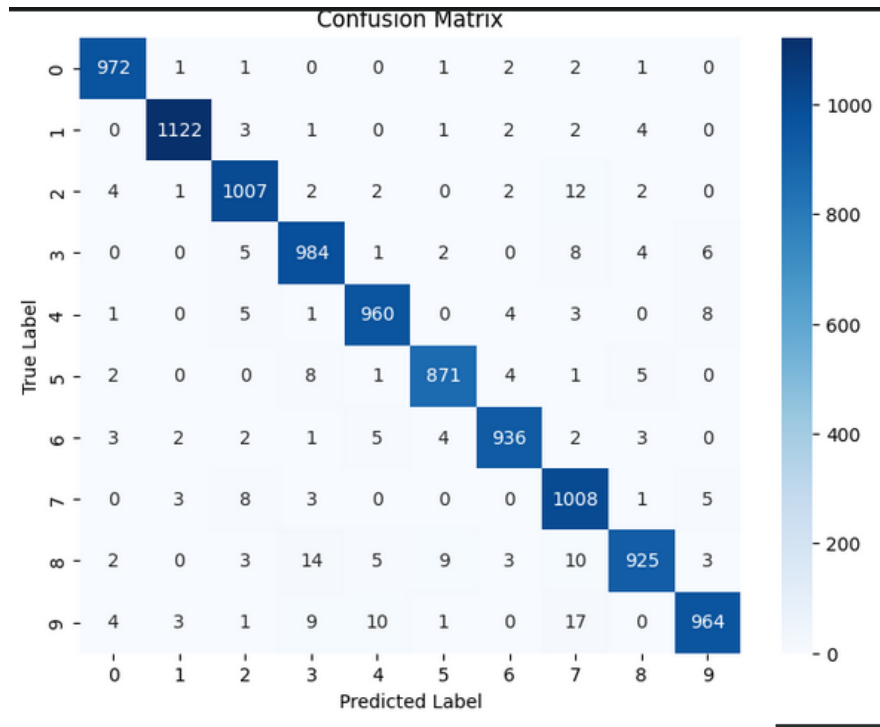


Figure 1.5: Confusion matrix of MNIST dataset Model

	precision	recall	f1-score	support
0	0.98	0.99	0.99	980
1	0.99	0.99	0.99	1135
2	0.97	0.98	0.97	1032
3	0.96	0.97	0.97	1010
4	0.98	0.98	0.98	982
5	0.98	0.98	0.98	892
6	0.98	0.98	0.98	958
7	0.95	0.98	0.96	1028
8	0.98	0.95	0.96	974
9	0.98	0.96	0.97	1009
accuracy			0.97	10000
macro avg	0.98	0.97	0.97	10000
weighted avg	0.98	0.97	0.97	10000

Figure 1.6: Classification Report of MNIST dataset Model

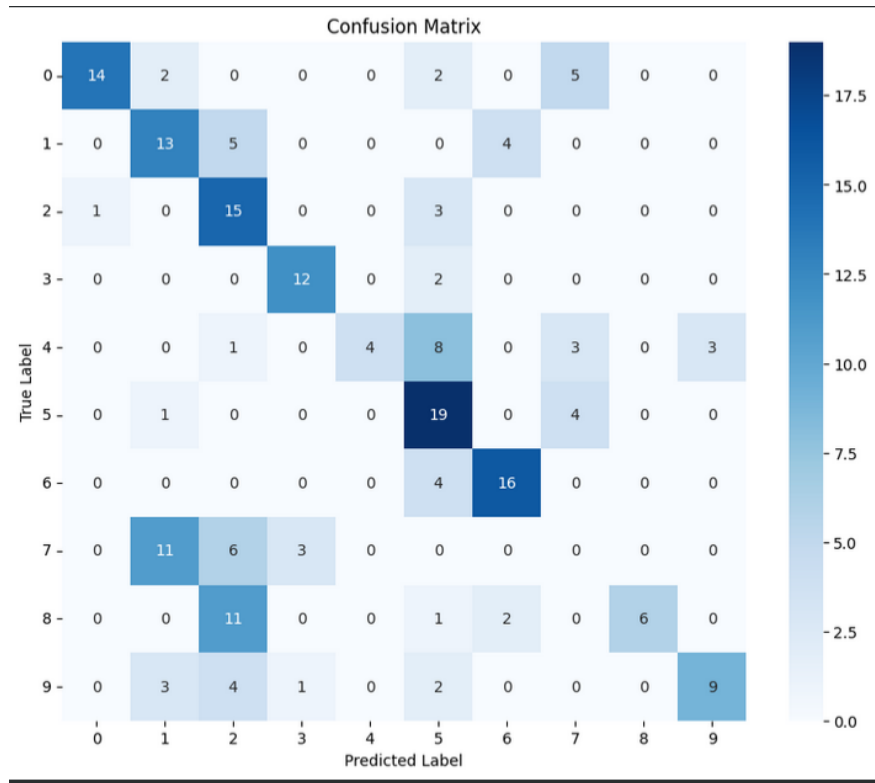


Figure 1.7: Confusion matrix of Our dataset Model

	precision	recall	f1-score	support
0	0.93	0.61	0.74	23
1	0.43	0.59	0.50	22
2	0.36	0.79	0.49	19
3	0.75	0.86	0.80	14
4	1.00	0.21	0.35	19
5	0.46	0.79	0.58	24
6	0.73	0.80	0.76	20
7	0.00	0.00	0.00	20
8	1.00	0.30	0.46	20
9	0.75	0.47	0.58	19
accuracy			0.54	200
macro avg	0.64	0.54	0.53	200
weighted avg	0.64	0.54	0.52	200

Figure 1.8: Classification Report of Our dataset Model

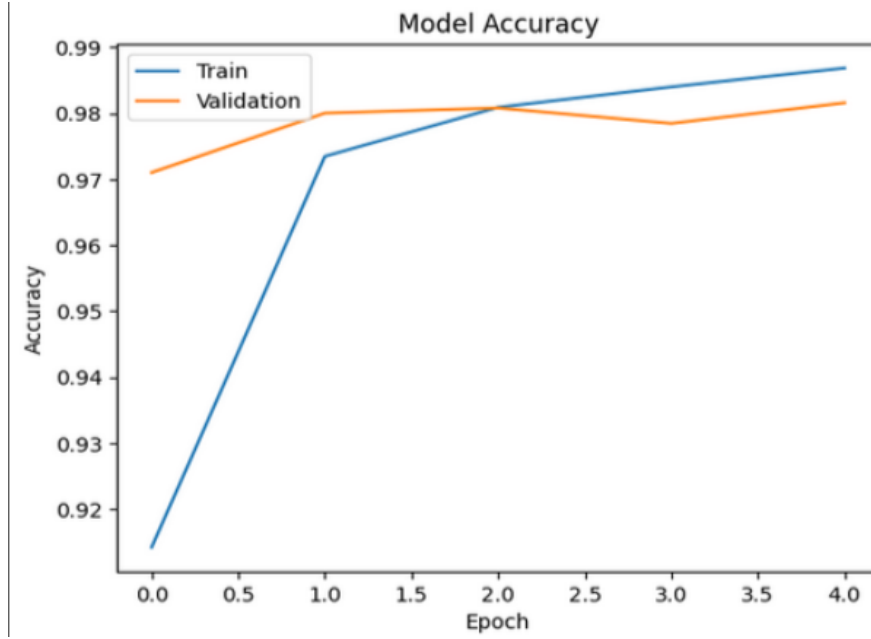


Figure 1.9: Training and Validation Accuracy

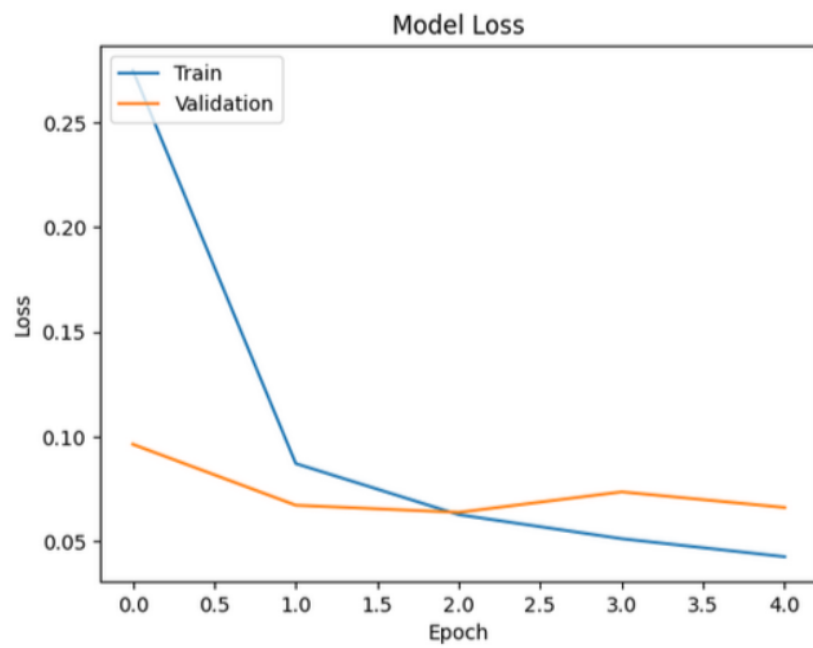


Figure 1.10: Training and Validation loss

Chapter 2

Conclusion

This project explores the application of convolutional neural networks (CNNs) for recognizing handwritten digits. Utilizing the well-established MNIST dataset and a custom-created dataset, we developed a model to evaluate its performance across diverse handwriting styles.

Key Achievements:

1. Validating CNNs' Capabilities:

The project demonstrates CNNs' effectiveness in analyzing and classifying image-based data. The model achieved high accuracy with the MNIST dataset, confirming the reliability of CNN architectures for this task.

2. Insights from Custom Dataset Performance:

The integration of a custom dataset highlighted limitations in the model's adaptability to less standardized data. Performance on the custom dataset was suboptimal, providing valuable insights into the need for enhanced preprocessing and training approaches when working with diverse handwriting styles.

3. Practical Learning Opportunities:

Implementing this project offered hands-on experience with CNN architectures, dataset preparation, model training, and optimization techniques, alongside lessons in addressing dataset-specific challenges.

Challenges and Insights Gained:

- The custom dataset posed challenges due to its variability and lack of preprocessing, resulting in lower model performance compared to MNIST.
- Issues like overfitting, hyperparameter tuning, and the need for data augmentation were addressed, emphasizing the importance of balancing dataset quality and diversity.
- These challenges underscored the necessity of rigorous data cleaning and preparation for practical applications.

Future Directions:

- **Enhancing Dataset Quality:**

Refining the custom dataset by preprocessing and augmenting data to reduce noise and variability.

- **Adapting the Model:**

Fine-tuning the CNN architecture or incorporating advanced techniques like transfer learning to improve performance on non-standard datasets.

- **Exploring Broader Applications:**

Extending these methodologies to other domains, such as character recognition or document analysis, while addressing the specific needs of each dataset.

This project successfully demonstrated the strength of CNNs for handwritten digit recognition using the MNIST dataset but revealed challenges when applied to more diverse handwriting styles in the custom dataset. These findings provide a strong foundation for improving adaptability in future work, contributing to a deeper understanding of CNNs' practical applications.

References

- [1] A. Challa, “Automatic Handwritten Digit Recognition On Document Images Using Machine Learning Methods,” 2019, pp. 50.
- [2] Nirase Fathima Abubacker, Mafas Raheem, ”Handwritten Digits Recognition,” Multimedia Systems, 2024.
- [3] J. S. Gaganashree and D. K. Padmashali, “Handwritten Digit Recognition Using Deep Learning,” International Journal of Research in Engineering, Science and Management, vol. 4, no. 7, 2021.
- [4] K. Y. Goh and A. S. Ghafar, “Handwritten Digit Recognition System Using Convolutional Neural Network (CNN),” PEAT, vol. 2, no. 1, 2021, pp. 578–592.
- [5] M. M. Ali and M. A. Hossain, “Recognition of Handwritten Digit Using Convolutional Neural Network (CNN),” Global Journal of Computer Science and Technology: Neural and Artificial Intelligence, vol. 19, no. 2, 2019, pp. 27–33.
- [6] Md. Anwar Hossain and Md. Mohon Ali, ”Optimizing Deep Learning Techniques for Accurate Handwritten Digit Classification,” vol. 19, version 1.0, 2019.
- [7] Vijayalaxmi R. Rudraswamimath and Bhavanishankar K, ”Comparative Analysis of CNN Architectures for Handwritten Digit Recognition,” vol. 4, issue 6, June 2019.
- [8] Shafaque Ahmareen, Alreem Khalid Khamies Alabdouli, and Sirisha Polturi, ”Machine Learning Applications in Education: A Case Study on Digit Recognition Systems,” 2024.

- [9] Shubham Mendapara, Krish Pabani, and Yash Paneliya, "Implementation of Handwritten Digit Recognition Using TensorFlow and Keras," vol. 7, issue 5, 2021.
- [10] "Implementation of Deep Learning—Handwritten Digits Recognition, Tensor-flow, CNN," from DeepLearning_by_PhDScholar, October 24, 2020.
- [11] Krish Naik, "Basic Understanding of Deep Learning."
- [12] Simon Haykin, *Neural Networks and Learning Machines*, McMaster University, Hamilton, Ontario, Canada.
- [13] D. C. Ciresan, U. Meier, and J. Schmidhuber, "Multi-column Deep Neural Networks for Image Classification," *arXiv*, 2012, arXiv:1202.2745.
- [14] X. X. Niu and C. Y. Suen, "A Novel Hybrid CNN–SVM Classifier for Recognizing Handwritten Digits," *Pattern Recognition*, vol. 45, 2012, pp. 1318–1325.
- [15] Nimisha Jain, Kumar Rahul, Ipshita Khamaru, Anish Kumar Jha, and Anupam Ghosh, "HandWritten Digit Recognition Using Convolutional Neural Network (CNN)," *International Journal of Innovations Advancement in Computer Science, IJIACS*, vol. 6, issue 5, 2017, ISSN 2347–8616.
- [16] Berend-Jan van der Zwaag, "Handwritten Digit Recognition: A Neural Network," *International Conference, 7th Fuzzy Days, Dortmund, Germany, October 1–3, 2001*.
- [17] S. M. Shamim, Mohammad Badrul Alam Miah, Angona Sarker, Masud Rana, and Abdullah Al Jobair, "Handwritten Digit Recognition Using Machine Learning Algorithms," *Global Journal of Computer Science and Technology: D Neural Artificial Intelligence*, vol. 18, issue 1, version 1.0, 2018.
- [18] S. Ahlawat and R. Rishi, "A Genetic Algorithm-Based Feature Selection for Handwritten Digit Recognition," *Recent Patents on Computer Science*, vol. 12, 2019, pp. 304–316.